RESOURCE_LIMITS

Container Resource Limits — Policy & Reference

> **One-line summary:** every container started under HelixAgent gets a hard memory cap, a swap cap equal to the memory cap, a pids cap, and a positive `oom_score_adj` so the container is the preferred OOM-killer victim — never the user's GUI session.

This document is the single source of truth for *why* the policy exists, *how* it is enforced at three independent layers, and *how* to tune it when adding a new container or stack.

---

1. The problem this policy solves

On the dev workstation that hosts the HelixAgent stack, the user GUI session has been observed to be torn down with `user@1000.service: Main process exited, code=killed, status=9/KILL` multiple times per day during heavy work sessions (Kimi CLI + Claude Code

- Podman compose stacks for Boba/MeTube + HelixAgent stacks).

Forensics from `journalctl`:

Date / Time	user-1000.slice memory peak	Outcome
2026-04-23 22:03	47.4 GiB	Clean stop
2026-04-25 02:46	**60.4 GiB** + 7.7 GiB swap	SIGKILL of user manager
2026-04-25 13:37	**60.5 GiB** + 4 GiB swap	SIGKILL of user manager

Box has 62 GiB RAM and 16 GiB swap. Without per-container limits, a runaway image build, a leaking MCP server, or a perfectly normal stack that just happens to overlap with three Claude Code sessions is enough to push the user slice past 60 GiB. Once that happens:

- btrfs + heavy paging stalls system threads.
- GDM/Wayland watchdog cannot reach gnome-shell within timeout.
- `systemd` SIGKILL's the user manager because its `TimeoutStopSec` expires while it itself is paging.
- User is dropped at the GDM greeter, all running tools die.

Important: the kernel OOM killer never fires — there are no `Out of memory: killed process …` lines in `journalctl -k`. So we cannot rely on the kernel to clean up. The policy below is what makes Linux do the right thing.

---

2. Defense in depth

Three layers, each independently sufficient to contain a runaway:

```
┌──────────────────────────────────────────────────────────────────────┐
│ Layer 3 — system-side oomd / cgroup parent limits (NOT used
│
│           on this host because we do not have sudo here)
│
└──────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────┐
│ Layer 2 — the `with-mem-cap` / `compose-up` user wrappers
│
│           in ~/.local/bin/. Run any heavy command inside a
│
│           transient `systemd-run --user --scope` with a hard
│
│           cap. If a container blows past Layer 1, Layer 2
│
│           catches it. Default 16G for `compose-up`.
│
└──────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────┐
│ Layer 1 — per-container caps in every compose file:
│
│           mem_limit, memswap_limit, pids_limit, oom_score_adj
│
│           Enforced by the kernel via cgroup v2 memory.max,
│
│           memory.swap.max, pids.max. THIS DOC describes Layer
```

1.

When all three layers are in place a memory leak in a single MCP server still kills only that MCP server, not the user session.

---

3. The four cap fields

Every service in every user-owned compose file has exactly these four top-level keys (Podman Compose / Docker Compose v2 schema):

Key	Type	Why
`mem_limit`	size string (`512m`, `2g`)	Hard cgroup `memory.max`. Kernel kills processes inside the cgroup that try to exceed it.
`memswap_limit`	size string ≥ `mem_limit`	`memory.swap.max`. We **set it equal to `mem_limit`** so the container cannot smuggle the leak into swap. Setting it higher just delays the eventual SIGKILL while thrashing the disk.
`pids_limit`	int (≥ 128)	`pids.max`. Stops fork bombs and runaway thread pools from filling the pid space.
`oom_score_adj`	int 100–1000	The Linux `oom_score_adj` for processes inside. Positive values mean "kill me first." We use `500` everywhere so kernel/oomd always picks a container over the user manager (which has `OOMScoreAdjust=-900`).

> **Why not `deploy.resources.limits`?** That key only takes effect under Docker Swarm. Podman Compose ignores it. The top-level form above works on both Docker (non-Swarm) and Podman, which is what we actually run.

---

4. The policy itself

The cap for a service is chosen by **first-match-wins glob matching against the lowercased service name**. The full table lives in two synchronized places:

- `Containers/scripts/resource-policy/policy.yaml` — used by the bulk cap-applier (`apply_caps.py`).

- `Containers/pkg/policy/policy.go::Default()` — used at runtime by Go callers that want to apply the same caps when starting a container programmatically.

4.1 Selected cap tiers (full list in source)

Service category	Pattern	mem	mem-swap	pids	Why
LLM inference	`ollama*`, `*triton*`, `vllm*`	12g	12g	2048	Model weights stay resident in RAM.
Browser MCP	`mcp-puppeteer*`, `mcp-playwright*`, `mcp-browserbase*`	3g	3g	4096	Headless Chromium needs many threads + RAM.
Image / ML MCP	`mcp-stable-diffusion*`, `mcp-imagesorcery*`	3–4g	same	1024	ML inference + image buffers.
Generic MCP	`mcp-*`	1g	1g	1024	Most are thin Node/Python wrappers.
SonarQube	`sonarqube*`	8g	8g	2048	Java + ES embedded.
Search	`elasticsearch*`, `opensearch*`	6g	6g	2048	JVM heap + overhead.
Vector DBs	`qdrant*`, `weaviate*`, `chroma*`	3–6g	same	1024	Index in RAM.
SQL DBs	`postgres*`, `mysql*`, `mongodb*`	4g	4g	1024	Buffer pools.
Caches	`redis*`, `memcached*`, `valkey*`	1g	1g	1024	Bounded-by-config.
Messaging	`kafka*`, `pulsar*`, `rabbitmq*`	2–4g	same	1024–2048	Page cache + JVM.
Reverse proxies	`nginx*`, `traefik*`, `caddy*`	512m	512m	1024	Stateless.
Helix application		3–4g	same	2048	Real workloads.

Service category	Pattern	mem	mem-swap	pids	Why
	`helixagent*`, `helixllm*`, `helixqa*`, `helix*`				
Build / CI	`*-builder*`, `*-ci-*`, `*-test*`	3–4g	same	2048–4096	Compilers fork heavily.
Default fallback	(no match)	2g	2g	1024	Conservative.

`oom_score_adj` is **always** `500` — see § 4.2.

4.2 Why `oom_score_adj = 500` everywhere

The user systemd manager (`user@1000.service`) is set to `OOMScoreAdjust=-900` by the system-level hardening in this project (or defaults to ~0 without it). We want every container in the user slice to be a more attractive OOM target than that manager. Setting `oom_score_adj = 500` produces an effective score of `oom_score_base + 500` which is virtually guaranteed to be picked over the user manager. Higher values (e.g. 1000) work too but can interact poorly with services that *should* survive a pressure spike (Postgres); 500 is the smallest value that still reliably saves the user session.

4.3 Adding a new service

Two cases:

1. **The service name matches an existing pattern** — nothing to do. Run `Containers/scripts/resource-policy/apply_caps.py` and the four keys will be inserted at the top of the new service block.

2. **It needs a custom cap (heavier than its pattern allows)** — add a rule to *both* `policy.yaml` and `policy.go::Default()`. Keep the ordering: more-specific patterns above less-specific ones.

Then run the test suite (§ 5). If `test_caps.py` passes, you're done.

---

5. Tooling

```
Containers/
└── scripts/resource-policy/
    ├── policy.yaml          # YAML rules (single source of truth
```

```
#1)
    ├── apply_caps.py        # bulk-applier (idempotent)
    ├── test_caps.py         # 20 tests covering matching,
coverage,
    │                        #   placement, budget, and
idempotency
    └── apply-report.md      # last run's summary
└── pkg/policy/
    ├── policy.go            # Go rules (single source of truth
#2)
    └── policy_test.go       # 6 Go tests
```

Run the bulk applier

```
cd HelixAgent
Containers/scripts/resource-policy/apply_caps.py [--dry-run]
```

It walks the project, skips third-party submodules, applies caps to every user-owned compose file. Idempotent on repeat runs.

Run the tests

```
cd HelixAgent
python3 Containers/scripts/resource-policy/test_caps.py
go test ./Containers/pkg/policy/...
```

Use the Go policy at runtime

```
import "digital.vasic.containers/pkg/policy"

p := policy.Default()
cap := p.CapFor(serviceName)            // returns Cap{Mem,
        Memswap, Pids, OOMAdj}
mem, swap, _ := cap.Bytes()             // raw bytes if you need
        them
if err := cap.Validate(); err != nil { ... }
```

`pkg/policy` is dependency-free; safe to vendor into other projects.

---

6. Skip-list — files we do NOT touch

`apply_caps.py` skips paths under any of:

- `/.git/`, `/node_modules/`, `/vendor/`
- `/.container-caps-backup-*` (its own backups)
- `/MCP/submodules/`, `/external/` (third-party MCP servers)

- `/cli_agents/{openhands,fauxpilot,gpt-engineer,claude-code-source,claude-plugins,postgres-mcp,kilo-code,roo-code,nanocoder,plandex,taskweaver,bridle,qwen-code,swe-agent}/`
- `/cli_agents/HelixCode/HelixCode/` (nested checkout)
- `/HelixQA/tools/opensource/` (third-party tools)
- `/mcp-servers/`, `/HelixCode/mcp-servers/`
- `/HelixLLM/docs/`, `/docs/research/`, `/docs/specs/`, `/docs/examples/`

Adding new third-party submodules? Extend `SKIP_PATH_FRAGMENTS` in `apply_caps.py`. Never modify a compose file in code we don't own.

---

7. Budget math

Host: 62 GiB RAM, 16 GiB swap. Reserved for kernel + GUI + AI agents: ~12 GiB. Container budget: **50 GiB**.

`test_caps.py::TestTotalBudget::test_single_file_fits_budget` enforces this for every non-profile compose file. Profile-heavy files (`docker-compose.mcp-full.yml`, `docker-compose.protocols.yml`, etc.) are exempt because the user only ever brings up a subset of services via `--profile`. The `compose-up` wrapper adds a 16 GiB systemd scope on top of any `podman compose up` invocation, so even an over-eager profile selection cannot exceed that ceiling.

---

8. Tuning when you know better

Three escape hatches:

1. **Per-service override in the compose file.** Just write the four keys yourself. `apply_caps.py` will leave existing values alone unless run with `--force-rewrite`.

2. **Per-pattern override in `policy.yaml`.** Add a more-specific pattern higher up than the existing rule.

3. **Disable Layer 2 for a one-off invocation.** Run `podman compose up` directly instead of `compose-up`. Discouraged — preferred alternative is to raise the wrapper's `MemoryMax` for that session: `with-mem-cap -m 24G -- podman compose up …`.

---

9. Verifying enforcement at runtime

```
# Start any service:
cd HelixAgent && compose-up up -d postgres
```

```
# Inspect its cgroup:
podman inspect helixagent-postgres -f '{{.HostConfig.Memory}}'
# → 4294967296   (4 GiB)

# Or directly via cgroup v2:
cgpath=$(podman inspect helixagent-postgres -f
        '{{.State.CgroupPath}}')
cat /sys/fs/cgroup${cgpath}/memory.max
# → 4294967296
cat /sys/fs/cgroup${cgpath}/memory.swap.max
# → 4294967296
cat /sys/fs/cgroup${cgpath}/pids.max
# → 1024
```

If `memory.max` is `max` (i.e. unlimited) the cap was not propagated. That has happened with podman-compose <1.0 and some old quadlet generators; upgrade if you see it.

---

10. Change log

- **2026-04-25** — Policy created. 443/443 user-owned services capped across 35 compose files in the HelixAgent project. Go counterpart in `pkg/policy`. Comprehensive test suite (20 Python + 6 Go).